

基于Harness Engineering架构的企业实战项目

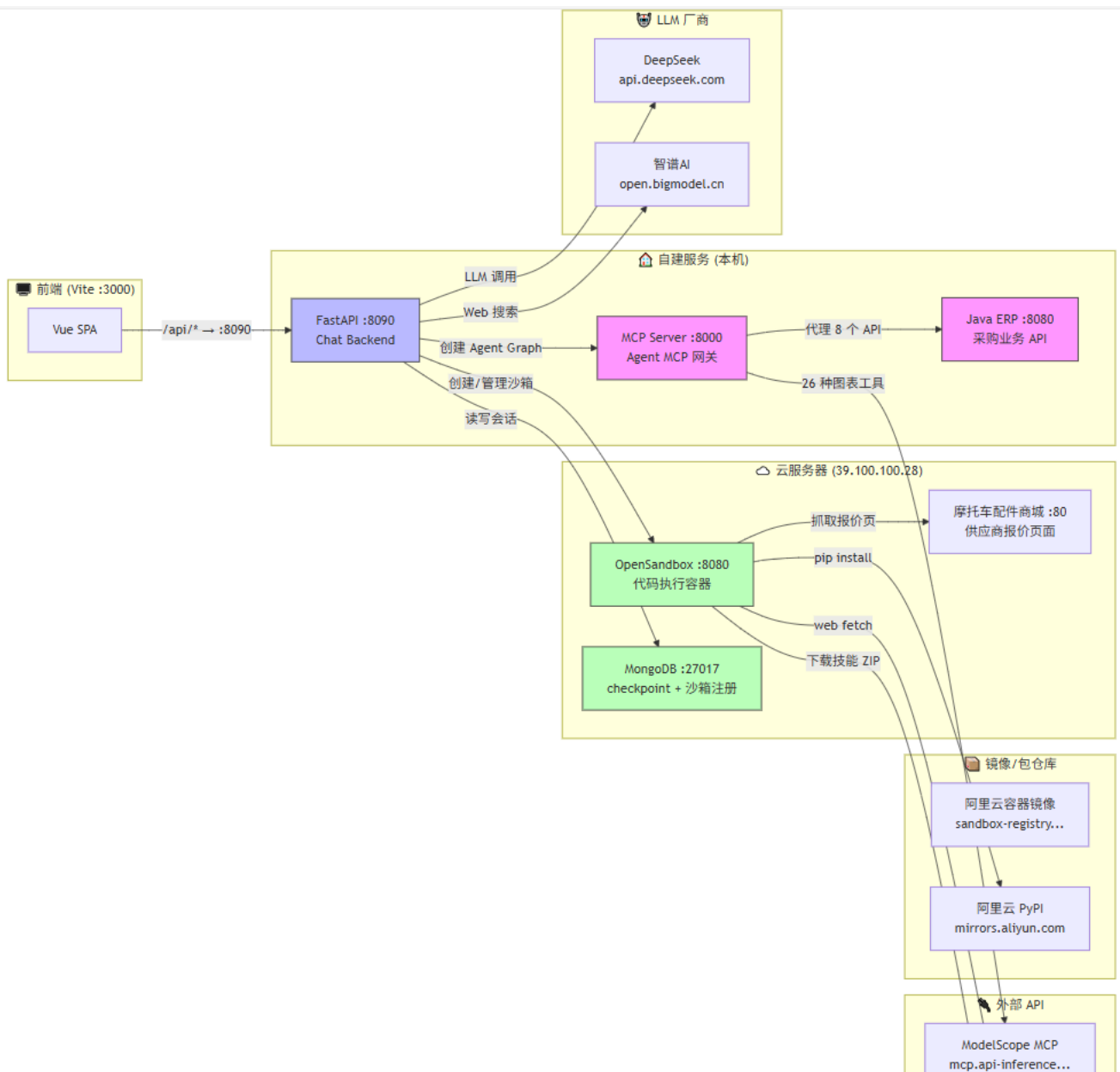
讲师：肖斌

如果进入直播间还没有拿到上课文档的找助教老师，今晚8点开始。不等人了



第一、项目总体介绍

1、项目外部依赖全景图



2、Agent核心代码架构

代码块

```

1  src/
2  |— api_view/
3  |   |— web_main.py
4  |   |— web_config.py
5  |   |— agent_loader.py
6  |   |— api/
7  |       |— chat.py
state、history
8  |       |— history.py
9  |
10 |— agent/
11 |   |— main_agent.py
precompute_agent_context()

```

🌐 Web 层 – FastAPI 对外服务

FastAPI 应用入口 (CORS、路由注册)

MongoDB 连接配置

AgentLoader 单例 (生命周期管理)

`/api/chat/stream (SSE)、resume、`

`/api/history CRUD + 展示消息存取`

🧠 Agent 层 – DeepAgent 核心

★ 主入口: `create_main_agent()` +

12			config.py	#	全局配置 (LLM、Store、Checkpoint、沙箱连接参数)
13			schema.py	#	数据模型 (ProcurementContext、UserPreferences、ChatRequest)
14			env_utils.py	#	环境变量加载 (.env → os.environ)
15			log_utils.py	#	日志工具
16			middleware_config.py	#	子 Agent 中间件工厂 (analyst / order)
17			mcp_tools_bean.py	#	MCP 工具分类 Bean
18					
19			memory/	#	 记忆模块
20			AGENTS.md	#	Agent 全局操作手册 (226 行)
21			prompts.py	#	主 Agent 系统提示词
22					
23			subagents/	#	 子 Agent 配置 (声明式)
24			loader.py	#	YAML 加载 + 工具名解析
25			configs/		
26			procurement_analyst.yaml	#	采购分析专家配置
27			procurement_order.yaml	#	采购订单专家配置
28					
29			middlewares/	#	 中间件栈 (8 个)
30			sandbox_health.py	#	1. 沙箱健康检查 + 自动恢复
31			context_injection.py	#	2. 用户上下文注入
32			skills_sync.py	#	3. 本地技能同步到沙箱
33			user_skills_restore.py	#	4. 持久化技能恢复
34			tools_summarization.py	#	5. compact_conversation 工具
35			memory_update.py	#	6. 用户偏好自动提取 + 持久化
36			sandbox_breaker.py	#	7. 沙箱熔断器
37			tool_error.py	#	(废弃 - ToolNode 已内置错误处理)
38					
39			tools/	#	 工具层
40			mcp_client.py	#	MCP 工具加载 (本地 ERP + ModelScope 图表)
41			chart_generator.py	#	26→1 图表工具合并
42			web_search.py	#	智谱搜狗 Web 搜索
43			hitl_tools.py	#	HITL 人工介入 (request_order_info)
44			assign_skill.py	#	技能分配 (下载→创建→测试→分配→持久化)
45			download_sandbox_file.py	#	沙箱文件下载到本地
46					
47			backends/	#	 沙箱后端
48			sandbox_manager.py	#	沙箱生命周期管理 (5 态: 预热→缓存→Mongo→新建)
49			sandbox_setup.py	#	沙箱创建 + Python 环境初始化
50			custom_opensandbox.py	#	OpenSandbox 封装 (注入 SANDBOX_PATH)
51			sandbox_proxy.py	#	代理层 (热替换, 18 个方法显式委托)
52					

```

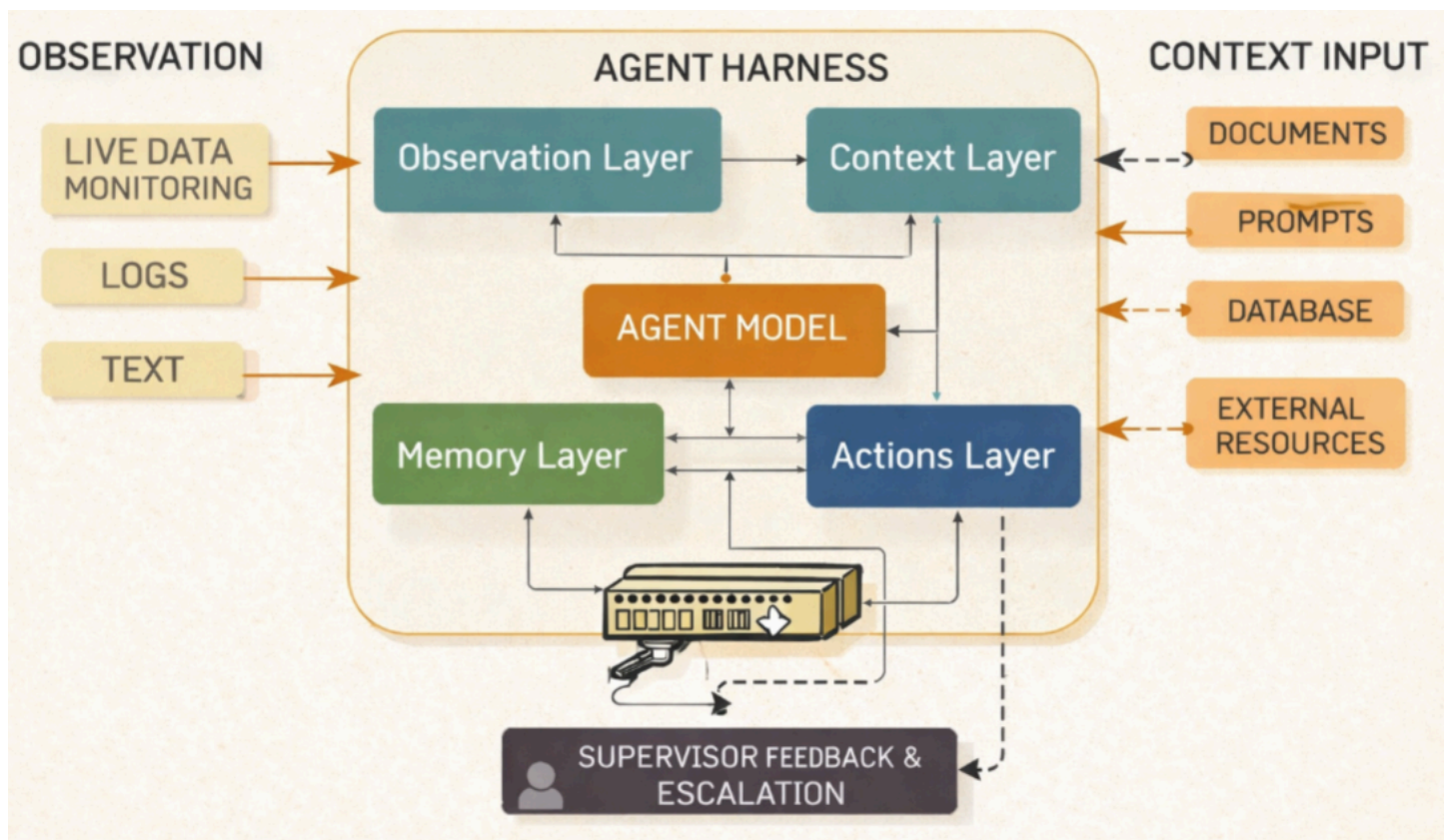
53 | └─ mcp_server/                                # 🔄 MCP 网关 – Agent ↔ Java ERP
54 |   └─ server_main.py                          #   MCP Server 入口 (FastMCP)
55 |   └─ server_config.py                       #   ERP 后端地址 + MCP 监听配置
56 |   └─ http_base.py                          #   httpx AsyncClient (连接池、超时)
57 |   └─ tools/
58 |     └─ suppliers_tools.py                  #   supplier_query MCP 工具
59 |     └─ parts_tools.py                      #   part_query / part_search /
part_by_supplier
60 |       └─ order_tools.py                    #   order_create / order_update /
order_search_details
61 |         └─ inventory_tools.py              #   inventory_warning MCP 工具
62 |
63 | └─ skills/                                  # 📖 技能库 (Markdown + Python, 渐进式加
载)
64 |   └─ main/
65 |     └─ skill-management/                  #   技能管理 SKILL.md +
download_skill.py
66 |       └─ procurement/
67 |         └─ procurement-analysis/          #   采购分析操作手册
68 |         └─ chart_params.md                #   26 种图表参数速查
69 |         └─ supplier-price-urls/           #   供应商报价 URL 映射
70 |         └─ web-scraper/                   #   沙箱内网页抓取工具
71 |         └─ web-content-fetcher/           #   HTML → Markdown 转换工具
72 |
73 | └─ test/                                    # 🧪 测试
74 |   └─ agent_test.py                        #   Agent 集成测试
75 |   └─ test_all_tools.py                    #   全工具测试
76 |   └─ test_mcp.py                          #   MCP 连接测试
77 |
78 | └─ download/                               # 📄 报告下载目录
79 |   └─ report_*.md                          #   Agent 生成的采购分析报告

```

第二、项目中具体Harness架构的实现

1、什么是 Harness Engineering?

- **Prompt Engineering** (2022-2024 主流) 主要优化单次交互的质量，重点是 “一句提示词该怎么写，模型才更容易给出想要的结果”
- **Context Engineering** (2025 年2月后兴起) 则动态构建知识、记忆、RAG，解决 “模型看什么” 的问题，减少幻觉、提高检索命中率
- **Harness Engineering** (当前阶段) 重点构建整个运行环境，解决 “模型怎么把长链路任务稳定做完” 的系统级问题。



Harness Engineering 是当前 AI Agent 开发领域的一个核心范式转变。它不再仅仅关注如何通过提示词（Prompt Engineering）或上下文管理（Context Engineering）来“优化模型输出”，而是聚焦于**构建包裹在模型之外的一整套系统化基础设施**，旨在将大语言模型（LLM）那种“不稳定、非确定性”的智能，转化为能够**稳定、可靠、长时间执行复杂任务**的工作引擎。

- **核心定义：** Harness（马具/缰绳）指的是除了模型本身之外的所有东西——工具、记忆、规划、安全护栏、执行循环、状态管理等。LangChain 团队将其精炼为公式：**Agent = Model + Harness**。模型提供“思考”能力，而 Harness 负责“让它真的能干活，而且别出大事”。
- **解决的问题：** 传统 Agent 在应对长周期、多步骤的复杂任务时，常面临上下文窗口爆炸、状态丢失、工具调用混乱、缺乏规划、无法从失败中恢复等工程难题。Harness Engineering 正是为了解决这些“工程上的‘稳’的问题”而生的。

一个完整的**Agent Harness**由几个**相互配合的核心能力层**构成，它们共同决定了Agent的行为和性能：

- **任务规划：** 通过内置的任务清单功能，将复杂目标分解为一系列可执行的子任务，并追踪每一步的执行状态。
- **虚拟文件系统：** 为Agent提供一个内置的虚拟文件管理系统，支持读写、编辑、搜索等操作，使其能像人类开发者一样进行工作。
- **子Agent委派：** 主Agent可以创建专门的“子Agent”来执行特定任务，实现任务隔离和上下文压缩。
- **上下文管理：** 通过文件系统等策略，管理长对话中的Token消耗，防止因上下文溢出而导致性能下降。

- 可插拔中间件：通过一组Hook函数，在模型和工具调用的关键节点注入自定义逻辑，实现缓存、重试、日志、PII检测等增强功能。
- Skills技能系统：将提示词、文档、脚本等打包为可复用的标准模块，为Agent注入特定领域的知识和能力。并做到自我进化。
- 持久化记忆：为Agent提供短期（对话历史）和长期记忆能力，使其能从历史交互中学习。

2、项目中Herness架构的核心能力(一)：长周期复杂任务的Planning能力

提供 `write_todos` 工具，Agent 可维护结构化任务列表（pending/in_progress/completed），持久化在 Agent state 中。

项目实现：

项目	说明
DeepAgents 内置	<code>write_todos</code> 工具由框架自动注册，无需显式配置
受益方	三个 Agent 全部受益：主 Agent 协调复杂任务、analyst 执行 5 步 order 执行 4 步订单操作
典型触发	analyst 启动时："ls → 读 SKILL.md → 收集数据 → 分析 → 图表 → 自动建议拆为 todo"
持久化	<code>CHECKPOINTER</code> (MongoDBSaver) 持久化 Agent state，跨重启

TOOL

`write_todos`

a3b92224-3090-a8ef-d6d4-80c9cb72d8c5

完成

参数

结果

任务清单

Updated todo list to [{ 'content': 'ERP内部数据收集：查火花塞零件信息和供应商', 'status': 'completed' }, { 'content': '外部数据采集：查 URL 映射、抓取报价页面或web搜索', 'status': 'in_progress' }, { 'content': '执行分析：价格、性能参数、性价比多维度对比', 'status': 'pending' }, { 'content': '生成图表（bar为主，辅以radar/waterfall等）', 'status': 'pending' }, { 'content': '生成最终报告 /analysis/report_{timestamp}.md', 'status': 'pending' }]

3、项目中Herness架构的核心能力(二)：安全的和动态路由的文件系统

项目实现：通过 `CompositeBackend` 实现三层路由文件系统。

路由分流效果：

路径前缀	后端	生命周期	隔离粒度
/AGENTS.md	OpenSandbox	沙箱生命周期	全局一份（只读）
/memories/{user_id}/	StoreBackend → MongoDB	跨会话永久	按 user_id 隔离
/persisted-skills/	StoreBackend → MongoDB	跨会话永久	按 namespace 组织
/skills/	OpenSandbox	沙箱生命周期	每次沙箱重建
/data/、 /analysis/	OpenSandbox	沙箱生命周期	临时工作区

代码块

```
1 backend = lambda rt: CompositeBackend(  
2     default=sandbox_backend,          # OpenSandbox (临时文件、代码执行)  
3     routes={  
4         "/memories/": StoreBackend(    # 用户偏好 → MongoDB 持久化  
5             runtime=rt,  
6             namespace=lambda rt: (getattr(rt.runtime.context, 'user_id',  
7 'default_user')),),  
8         "/persisted-skills/": StoreBackend( # 持久化技能 → MongoDB  
9             runtime=rt,  
10            namespace=lambda rt: SKILLS_STORE_NAMESPACE,  
11        ),  
12    },  
13 )
```

4、项目中Herness架构的核心能力(三)： Subagent的任务委派

主 Agent 拥有 `task` 工具，每次调用创建全新 Agent 实例，独立上下文，执行完返回单个报告。支持并行执行和特殊化配置。

项目实现： **Multi-Agent**

子 Agent 配置加载： loader.py

代码块

```
1  YAML 配置文件 (configs/*.yaml)  
2  | load_subagent_configs()      ← YAML → dict 列表  
3  | resolve_subagent_tools()    ← 工具名子串匹配 → 可调用对象  
4  | _validate_subagent_config() ← 必填字段校验  
5  ▼
```

两个子 Agent 对比：

	procurement-analyst	procurement-c
YAML	procurement_analyst.yaml	procurement_order.yam
description	含触发关键词 ("分析"、"对比"、"报告"...)	含触发关键词 ("下单"、"单"、"修改"...)
工具数	8 个 (6 MCP + web_search + generate_visualization)	5 个 (3 MCP + web_search request_order_info)
技能 scope	/skills/procurement/	/skills/order/
中间件	Summarization + 调用限制(50/200)	调用限制(20/50)
interrupt_on	无	order_create: approve/reorder_update: approve/r
context 隔离	独立上下文，analyst 长篇工具结果不污染主 Agent	独立上下文，订单操作不干

委派模板（AGENTS.md:74-121）：

主 Agent 的 AGENTS.md 为每个子 Agent 定义了结构化的 task 调用模板，确保 user_id/username/偏好 正确传递到子 Agent。

5、项目中Herness架构的核心能力(四)： Context Management — 上下文管理

四种策略——Input context（启动时加载）、Compression（自动 offload + summarization）、Isolation（子 Agent 隔离）、Long-term memory（跨线程持久化）。

项目实施：

a. Input Context

输入	来源	何时加载
system_prompt	prompts.py	Agent 创建时
AGENTS.md	本地 → 沙箱 /AGENTS.md (main_agent.py:122-124)	Agent 创建时，memory 中
用户偏好	/memories/{user_id}/preferences.md (StoreBackend)	每轮对话前 read_file
技能 frontmatter	各 SKILL.md 的 YAML frontmatter (渐进式披露)	Agent 启动时扫描
用户上下文	ContextInjectionMiddleware 注入 SystemMessage	每轮 before_agent
子Agent报告路径/订单确认	委派结果	收到返回后

b. Compression

机制	实现	i
自动 Offloading	DeepAgents 内置，工具结果 > 20k tokens → 写入文件，上下文只保留路径 +前10行预览	框架默认
自动 Summarization	SummarizationMiddleware，上下文 85% 窗口 → 自动压缩历史为摘要，完整历史保存到 /conversation_history/	tools_summariza
手动 compact_conversation	SummarizationToolMiddleware 注册的工具，Agent 在收到子Agent长篇报告/对话超6轮/用户频繁切换话题时主动调用	tools_summariza model=deepseek

c. Isolation（子 Agent 上下文隔离）

analyst 执行 5 步分析流程时会产生大量中间结果（MCP 返回的供应商/零部件 JSON、Python 脚本输出、图表生成参数等），全部隔离在子 Agent 自己的上下文窗口中。主 Agent 只收到最终的结构化报告。

d. Long-term Memory


```
25 | → store.aput() 持久化到 StoreBackend |
26 | → UserSkillsRestoreMiddleware 恢复（下次沙箱创建时） |
27 | → 清理压缩包 |
28 |_____|
```

基于SkillsSyncMiddleware发现链

代码块

```
1  yaml system_prompt: "ls /skills/procurement/ 扫描可用技能"
2    → 返回: chart_params.md, procurement-analysis/, supplier-price-urls/, web-
   scraper/
3    → read_file("procurement-analysis/SKILL.md") 加载操作手册
4    → 按流程激活各技能
```

7、项目中Herness架构的核心能力(六): Human-in-the-Loop — 人工介入

通过 `interrupt_on` 参数选择性地在工具调用前暂停，等待人工审批或修改。

项目实施：双层中断体系。

代码块

```
1  第1层：数据补充 (tool-internal interrupt)
2    request_order_info 工具内部调用 interrupt()
3    └─ 触发时机：Schema校验发现必填字段缺失
4    └─ 恢复格式：{"supplement": "自由文本补充信息"}
5    └─ 循环：校验→补充→解析→校验...直到完整
6
7  第2层：最终审批 (interrupt_on 配置)
8    order_create / order_update 的 interrupt_on 配置
9    └─ 触发时机：数据完整，准备执行
10   └─ 恢复格式：{"decisions": [{"type": "approve"|"reject"}]}
11   └─ 特性：approve → 执行 / reject → 取消
```

代码位置：


```
5    【自动路径】对话结束后
6    → MemoryUpdateMiddleware.aafter_agent()
7    → LLM 提取 {suppliers: [...], query: "..."}
8    → store.aput() 写入 StoreBackend
9    → 跨会话保留
```

9、项目的最大安全问题： 基于OpenSandbox来解决

OpenSandbox 就是这个项目的**唯一安全边界**。所有 Agent 行为——运行 Python 脚本、执行 Shell 命令、读写文件、下载，新建技能（skill）、爬取网页——全部发生在这个容器内部。**OpenSandbox 的配置决定了这道墙有多厚。**

综合风险评估

风险	等级	影响	当前状态
沙箱无网络出口限制	高	数据外传、内网探测、云凭证窃取	network_policy 解决
无 URL 白名单 (download_skill)	中	从恶意服务器下载	network_policy 解决
skill_name 路径穿越	中	assign_skill 访问任意沙箱路径	文件操作
user_id 隔离弱	中	多用户数据可被跨用户访问	文件操作
AGENTS.md 无技术只读	低	Agent 可能修改全局准则	防止文件修改
无边载配额/进程限制	低	沙箱资源耗尽	docker容器可以随时增
download_sandbox_file 无敏感文件过滤	低	沙箱文件可外传到本地	无限制

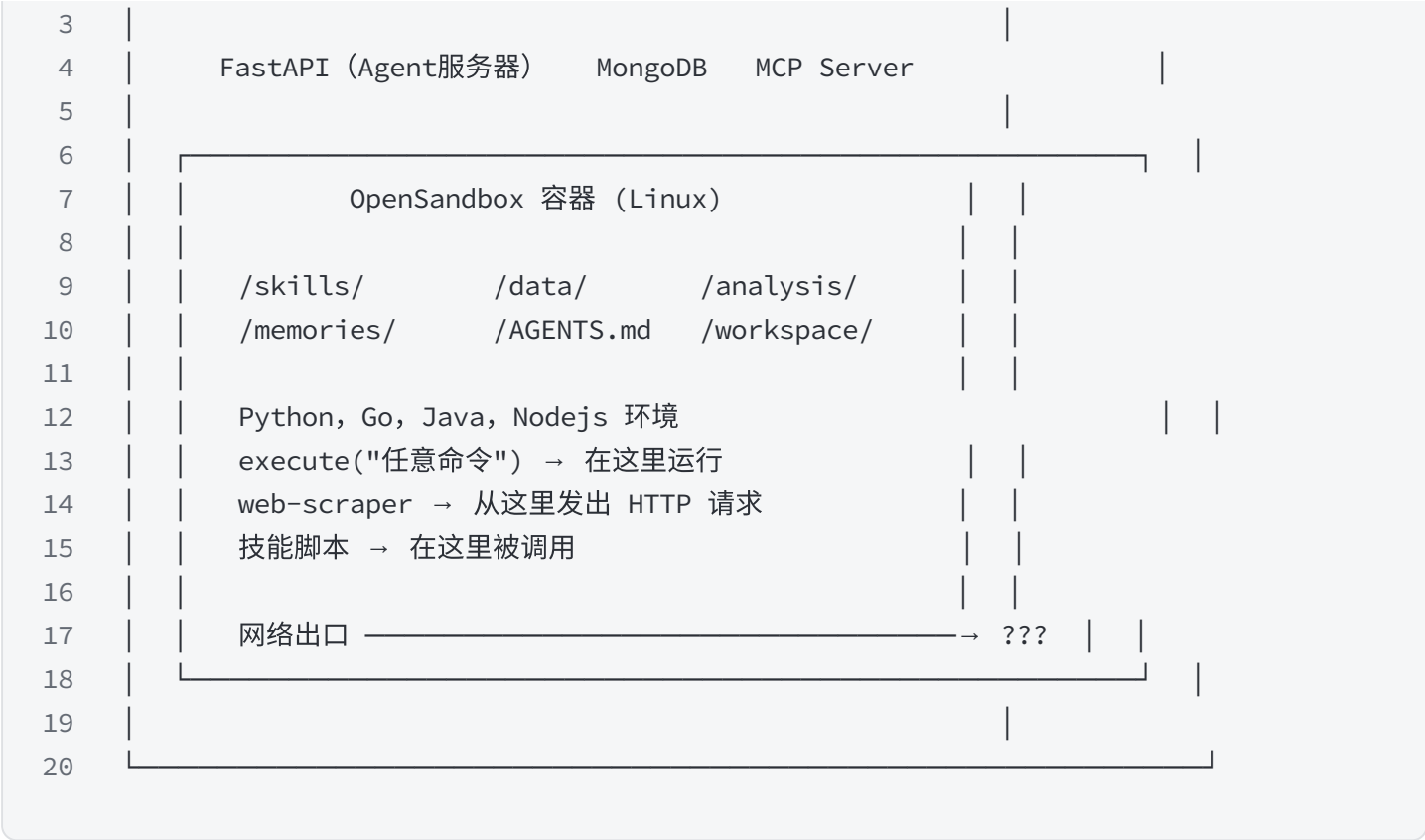
OpenSandbox 在这里扮演的角色：它是最后的兜底线。即使Skills全部失守——恶意技能被下载、解压、恶意代码、文件操作——第 5 阶段执行时，恶意代码仍然被关在容器里。**容器的网络策略决定了恶意代码能造成多大的外部影响。**

Sandbox 做边界定义：墙内 vs 墙外

代码块

```
graph LR; 1 --- 2; 2 --- Host[宿主机 (Windows)];
```

1		
2	宿主机 (Windows)	



OpenSandbox 防护了三件事：

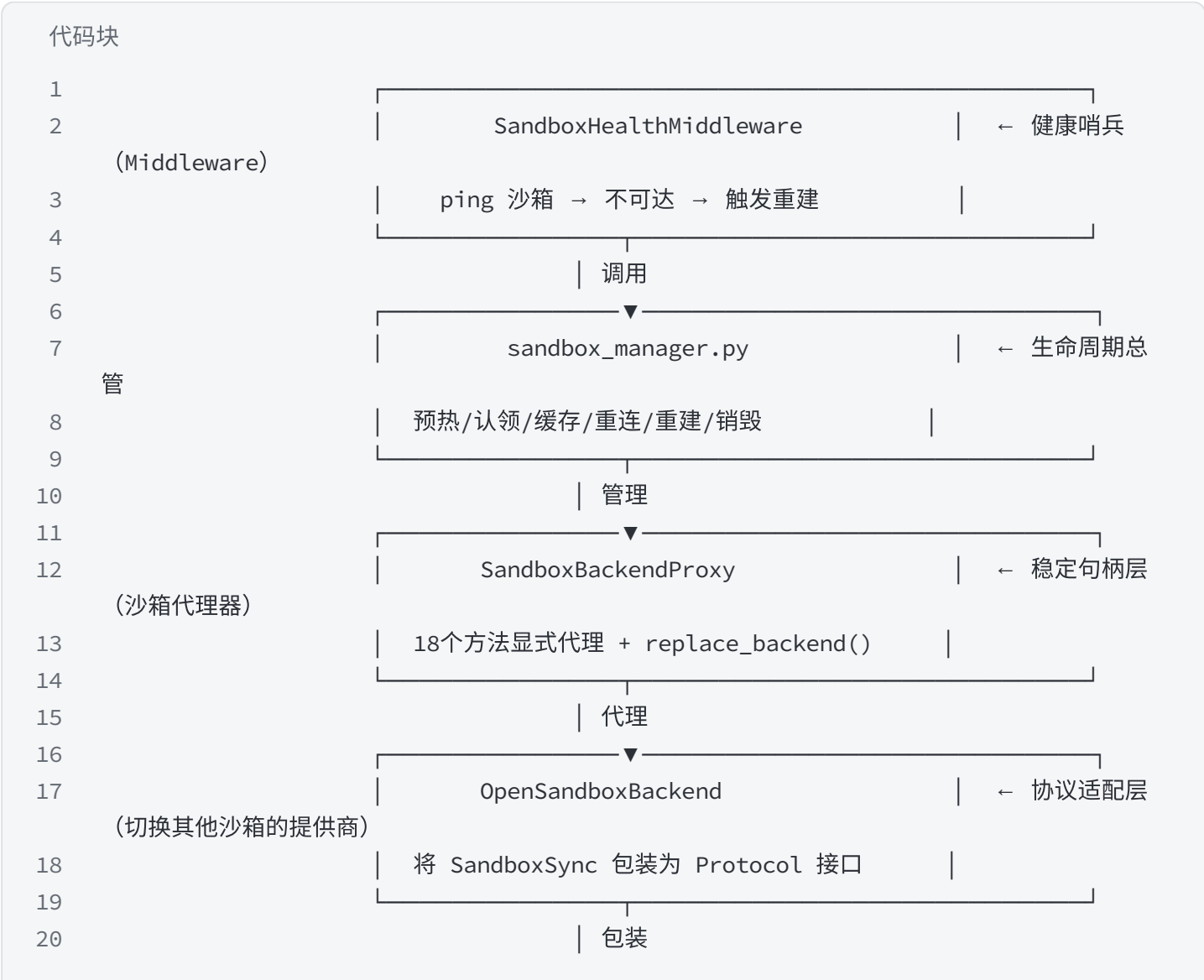
防护	机制
宿主机文件系统不可见	容器内 <code>/skills/</code> 、 <code>/data/</code> 都是容器自己的文件系统 宿主机 <code>E:\my_project\</code> 物理隔离
进程隔离	<code>execute()</code> 启动的进程在容器内运行，无法直接 主机进程
资源限制	<code>cpu: "2"</code> 、 <code>memory: "4Gi"</code> 防止容器耗尽宿主机

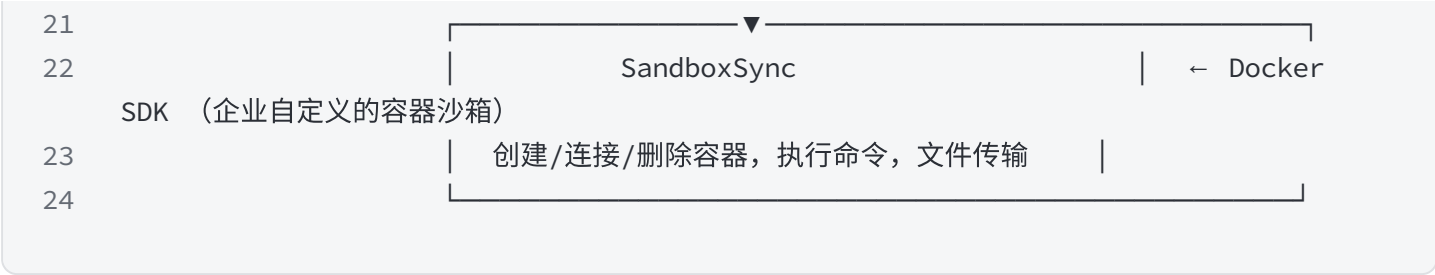
第三、项目亮点

1、用户作用域的沙箱隔离和沙箱预热机制

一、沙箱容器组件的层次关系

维度	SandboxSync	OpenSandboxBackend	SandboxBackendProxy	
来源	opensandbox 外部SDK	custom_opensandbox.py	sandbox_proxy.py	S y
层级	基础设施层	协议适配层	代理层	生
职责	Docker容器CRUD	将 SandboxSync 包装为 Protocol 接口	稳定句柄，透明代理+热替换	用
实例数量	N（每个沙箱1个）	N（与SandboxSync 1:1）	N（与用户1:1）	1
是否持有引用	—	持有 SandboxSync	持有 OpenSandboxBackend	持
生命周期	容器级（2小时超时）	与 SandboxSync 相同	跨沙箱重建（热替换）	全
热替换感知	✗ 不感知	✗ 不感知	✓ <code>replace_backend()</code>	✓ re
asyncio.to_thread	同步API	同步API	同步API（无异步逻辑）	✓



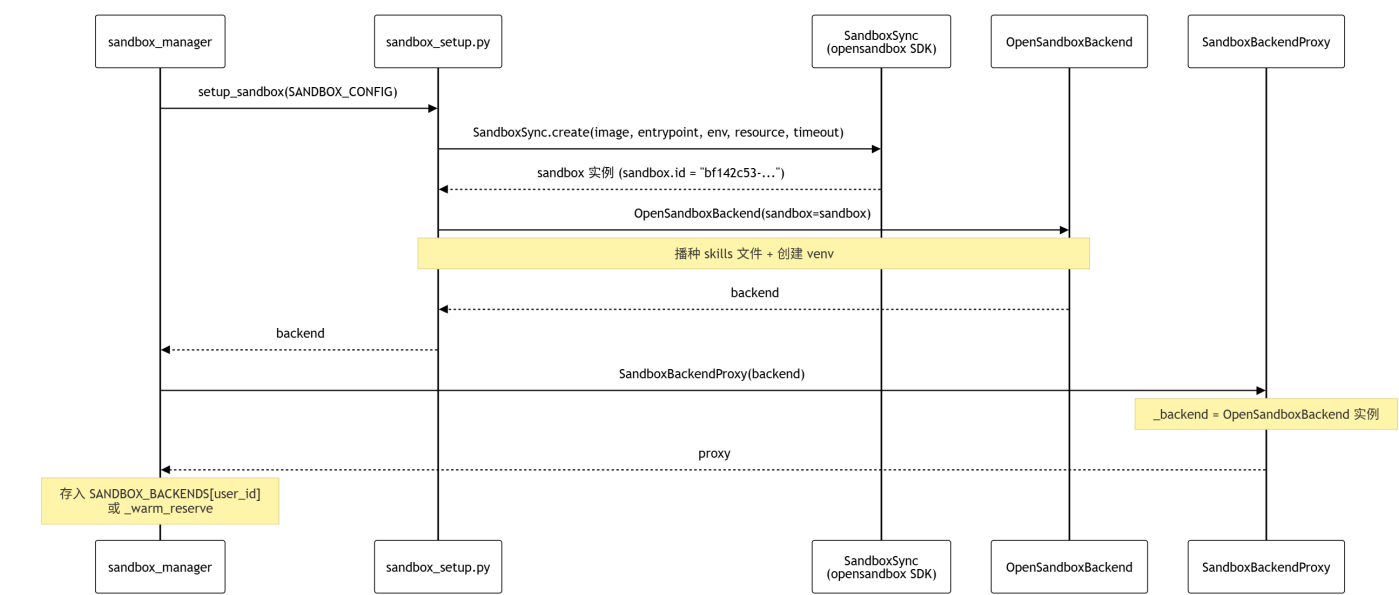


其他沙箱的提供商：

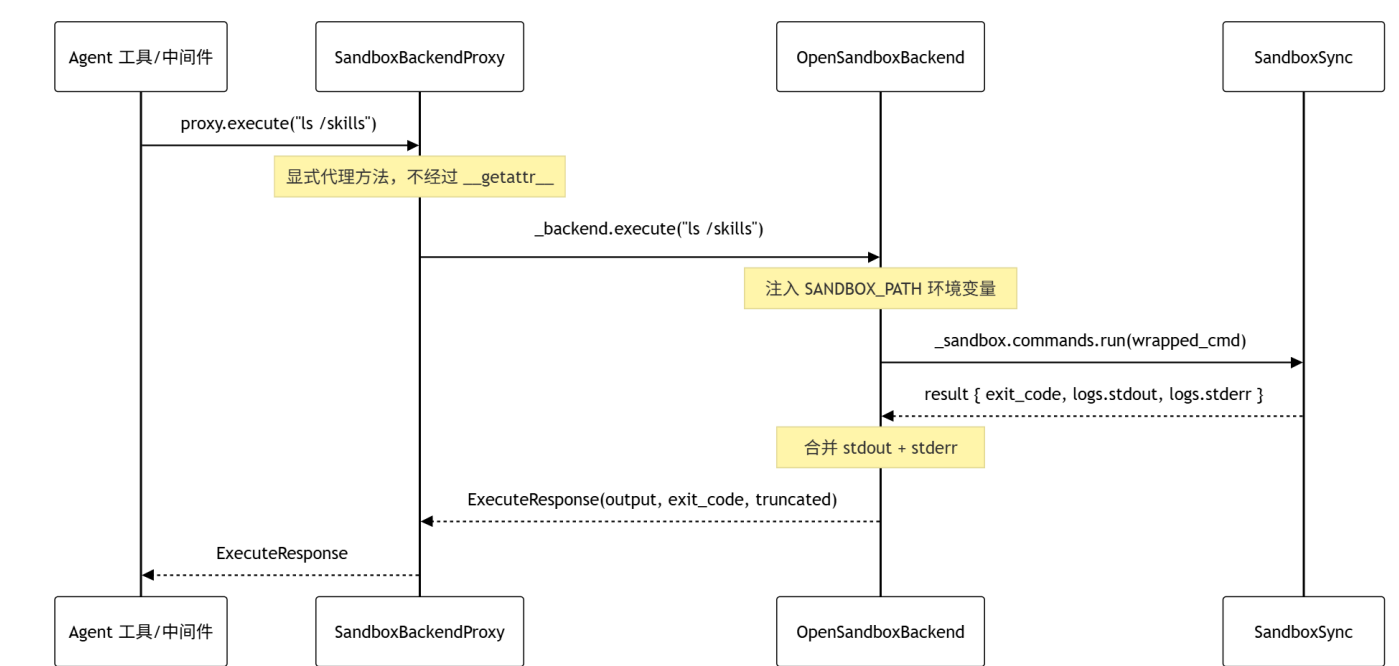
对比维度	Daytona	Modal	Runloop
公司/所属组织	Daytona（公司） 创始人：Ivan Burazin（CEO）、Vedran Jukić（CTO）等	Modal Labs（公司） 联合创始人：Erik Bernhardsson（CEO）、Akshat Bubna（CTO）	Runloop（公司） 联合创始人：Jonathan Wall（CEO）等
核心定位	极速启动的AI代码执行沙箱	强大的GPU计算和ML任务平台	为AI编码代理设计的用开发环境
融资情况	完成2400万美元A轮融资，由FirstMark Capital领投	完成3.55亿美元C轮融资，估值46.5亿美元	完成700万美元种子轮融资，由The General Partnership领投
总部/所在地	克罗地亚（Croatian-founded）	美国纽约市	美国旧金山
主要产品/平台	Daytona Sandbox	Modal平台	Runloop Devboxes
生命周期	持久化沙箱，支持自动停止/归档/删除	按需使用，资源随任务创建和销毁	按需创建的临时（ephemeral）开发境（devbox）
启动速度	极快，约90毫秒	较快，为serverless环境优化	面向代理优化，但具数据未明确
核心优势	启动速度最快、支持多语言、开源可自托管	强大的GPU支持（H100等）、Python原生、serverless	专为编码代理设计、供评估工具、与GitHub集成
计费模式	按需付费（计算/内存/存储），按秒计费	按需付费（GPU/CPU/内存），价格下调高达65%	基础套餐\$25/月 + 按用量计费
适用场景	高并发、需要快速响应的AI代码生成场景	机器学习模型训练、数据批处理等计算密集型任务	AI编码代理的测试、估和规模化部署
开放性	开源，可自托管	闭源，仅限云服务	闭源，仅限云服务
备注	从开发环境管理转型	近期完成8700万美元B轮融资，估值超10亿美元	专注于AI编码代理赛

二、本项目中沙箱核心流程

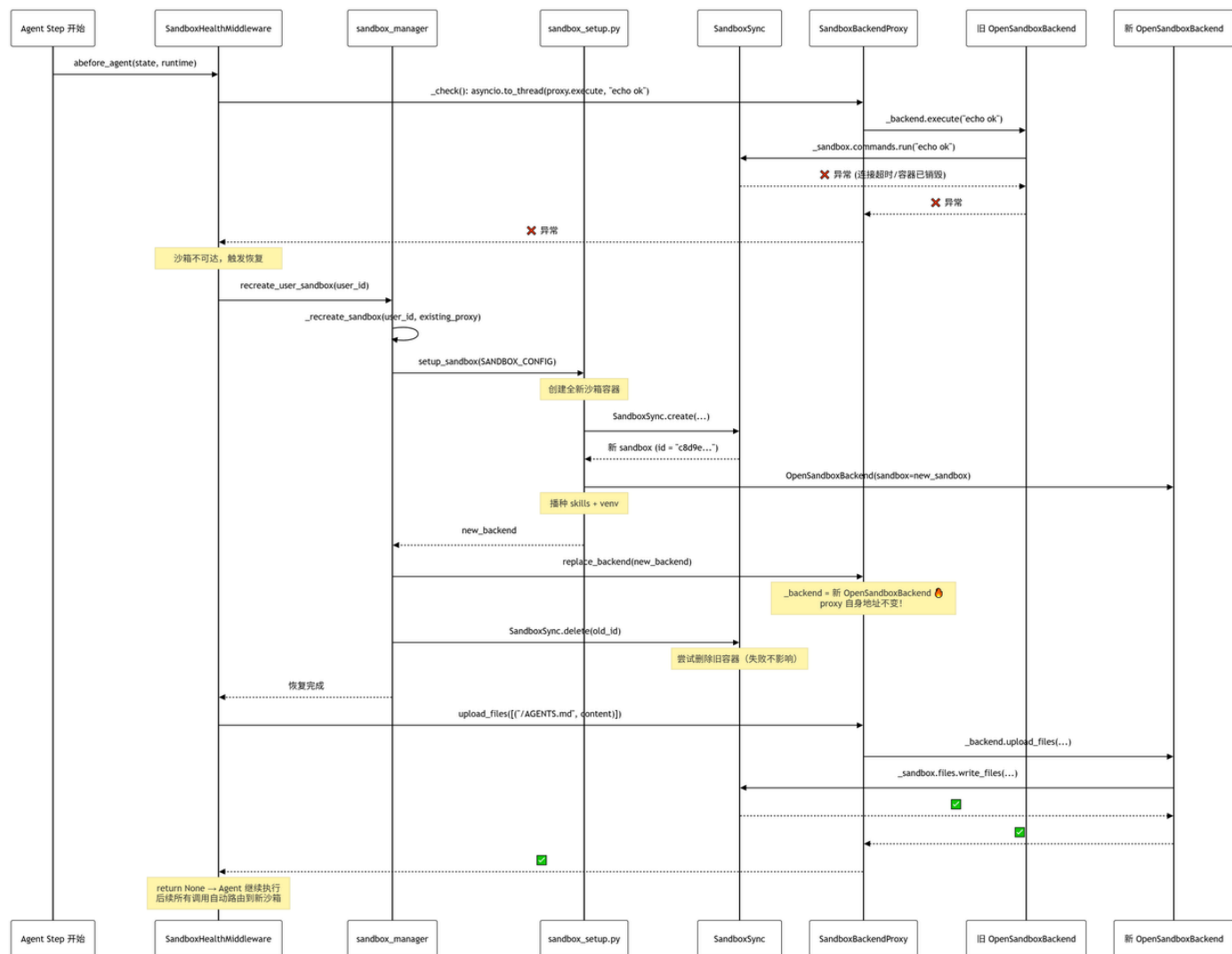
2.1 沙箱创建链路



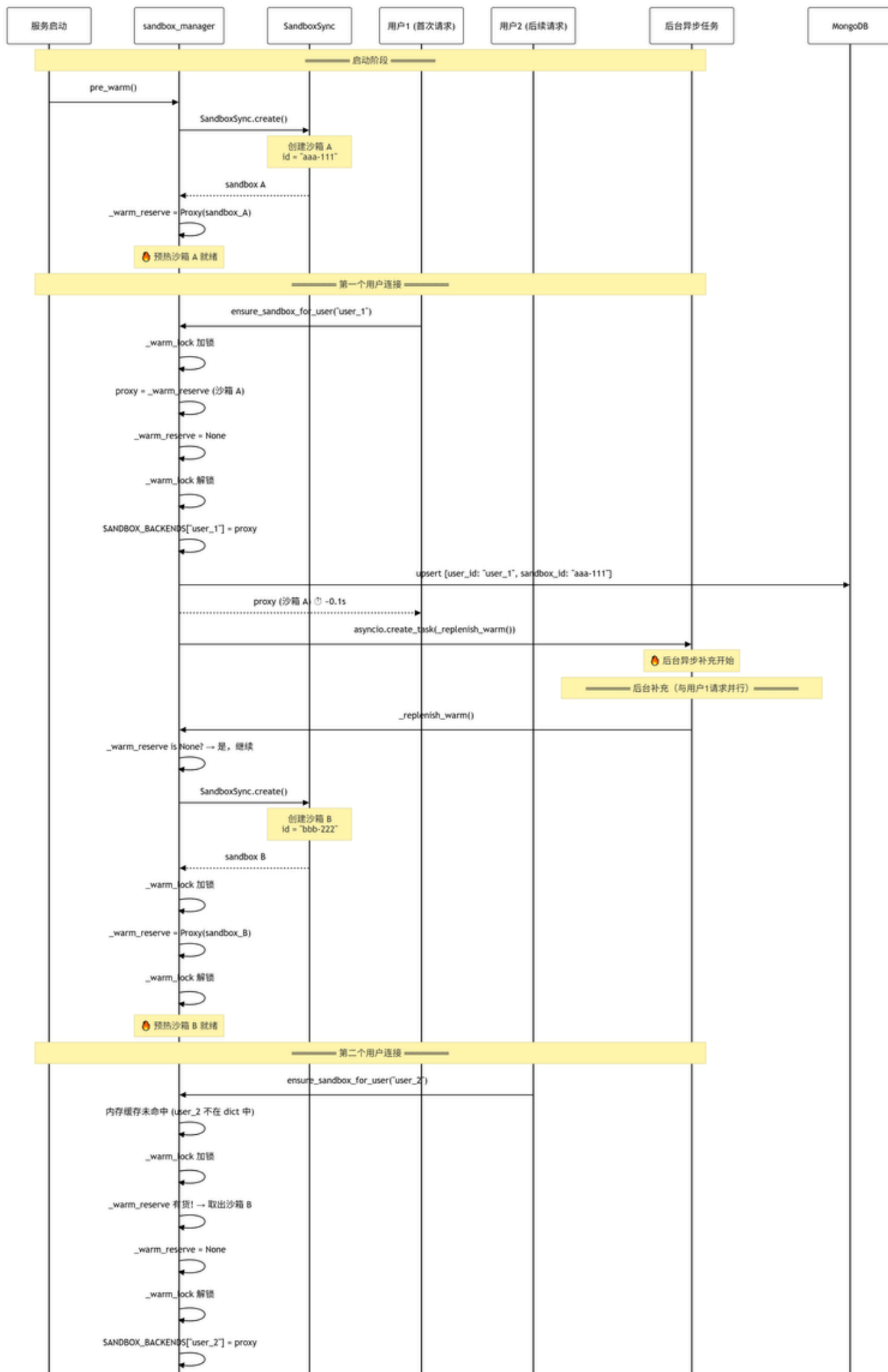
2.2 命令执行链路（正常路径）

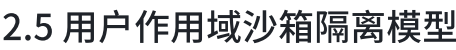


2.3 沙箱故障 → 热替换恢复链路



2.4 预热-认领-补充 时序图





2、中间件矩阵：故障全场景覆盖

一、中间件总览表

用户自定义 + 框架内置 + Agent自动添加

代码块

1	
2	create_deep_agent() Agent自动自动添加
3	
4	TodoListMiddleware → write_todos 任务管理工具
5	MemoryMiddleware → AGENTS.md 注入到 system prompt
6	SkillsMiddleware → 技能渐进式披露
7	FilesystemMiddleware → ls/read/write/edit/glob/grep/execute 工具
8	SubAgentMiddleware → task 子 Agent 调度工具
9	SummarizationMiddleware → 85% 自动摘要保底
10	AnthropicPromptCachingMiddleware → 提示缓存
11	PatchToolCallsMiddleware → 孤儿 tool_call 修复
12	
13	
14	用户 main_middleware 列表 (每次请求注入)
15	
16	1. SandboxHealthMiddleware ← NEW 自定义
17	2. ContextInjectionMiddleware ← NEW 自定义
18	3. SkillsSyncMiddleware ← NEW 自定义
19	4. UserSkillsRestoreMiddleware ← NEW 自定义
20	5. SummarizationToolMiddleware ← DeepAgents 内置, 手动启用
21	6. MemoryUpdateMiddleware ← NEW 自定义
22	7. SandboxCircuitBreakerMiddleware ← NEW 自定义
23	8. ModelCallLimitMiddleware ← LangChain 内置
24	9. ToolCallLimitMiddleware ← LangChain 内置
25	

#	中间件	来源	使用的钩子	
1	SandboxHealthMiddleware	自定义	abefore_agent	ping 检测
2	ContextInjectionMiddleware	自定义	before_agent, abefore_agent	注入上下文
3	SkillsSyncMiddleware	自定义	before_agent, abefore_agent	本地技能同步
4	UserSkillsRestoreMiddleware	自定义	abefore_agent	Store 恢复技能
5	SummarizationToolMiddleware	自定义	wrap_model_call	提供摘要能力
6	MemoryUpdateMiddleware	自定义	aafter_agent	提取对话记忆
7	SandboxCircuitBreakerMiddleware	自定义	before_model, abefore_model	连续熔断保护
8	ModelCallLimitMiddleware	LangChain 内置	before_model, after_model	模型调用限制
9	ToolCallLimitMiddleware	LangChain 内置	before_agent, after_agent, before_model, after_model, wrap_tool_call	工具调用限制

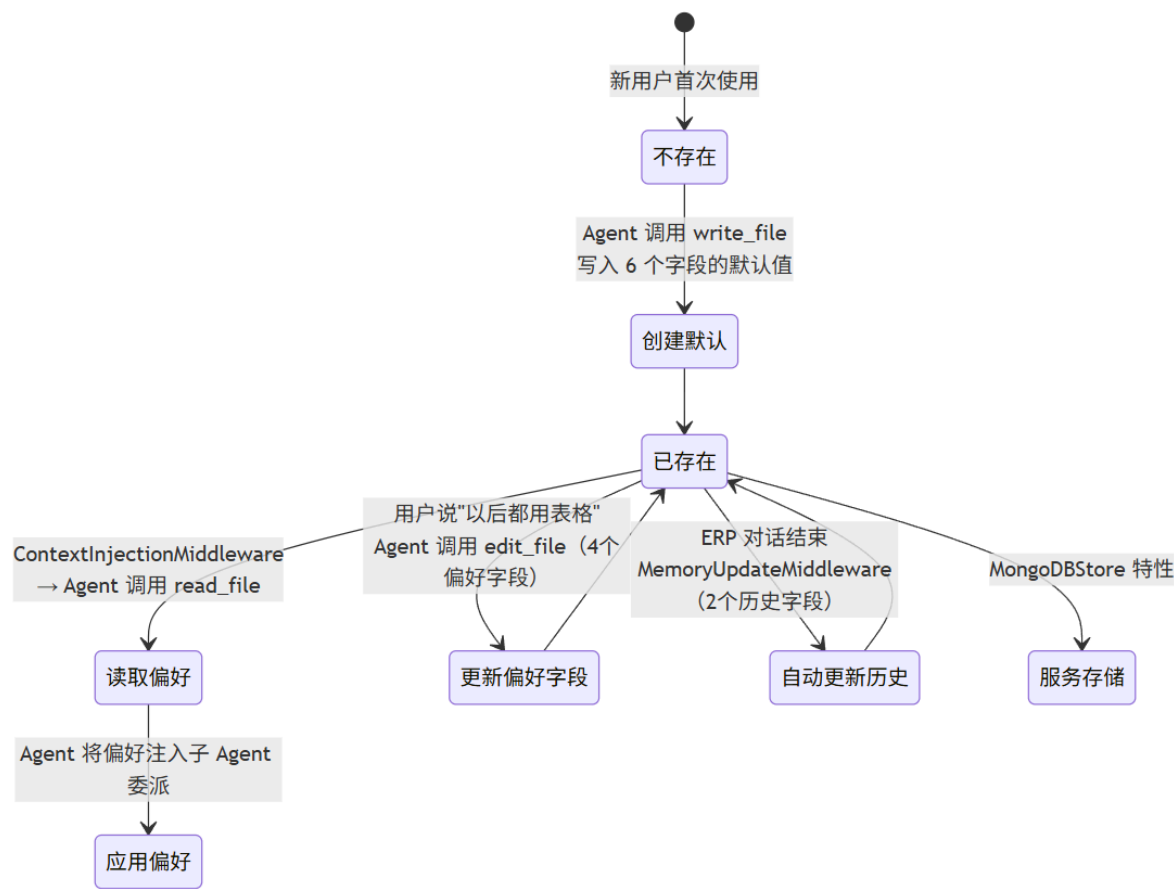
二、故障场景覆盖矩阵

故障场景	处理中间件	行为
沙箱超时/容器被销毁	#1 SandboxHealth	ping 检测 → 自动重建 → 继续
沙箱服务宕机（重建也失败）	#1 → #7 SandboxBreaker	重建尝试 → 连续失败 → 熔断
工具调用抛异常	ToolNode（框架内置）	捕获 → ToolMessage(status="error")
上下文溢出	#5 SummarizationTool + 框架 Summarization	自动摘要 → Agent自动判断摘要时机
LLM 陷入循环（50+ 次调用）	#8 ModelCallLimit	强制终止
工具调用风暴（200+ 次）	#9 ToolCallLimit	强制终止
用户记忆丢失	#6 MemoryUpdate	每次对话后自动写回 StoreBackend
沙箱丢失技能文件	#3 SkillsSync + #4 UserSkillsRestore	启动时自动恢复
用户身份未注入	#2 ContextInjection	首次 step 注入 SystemMessage

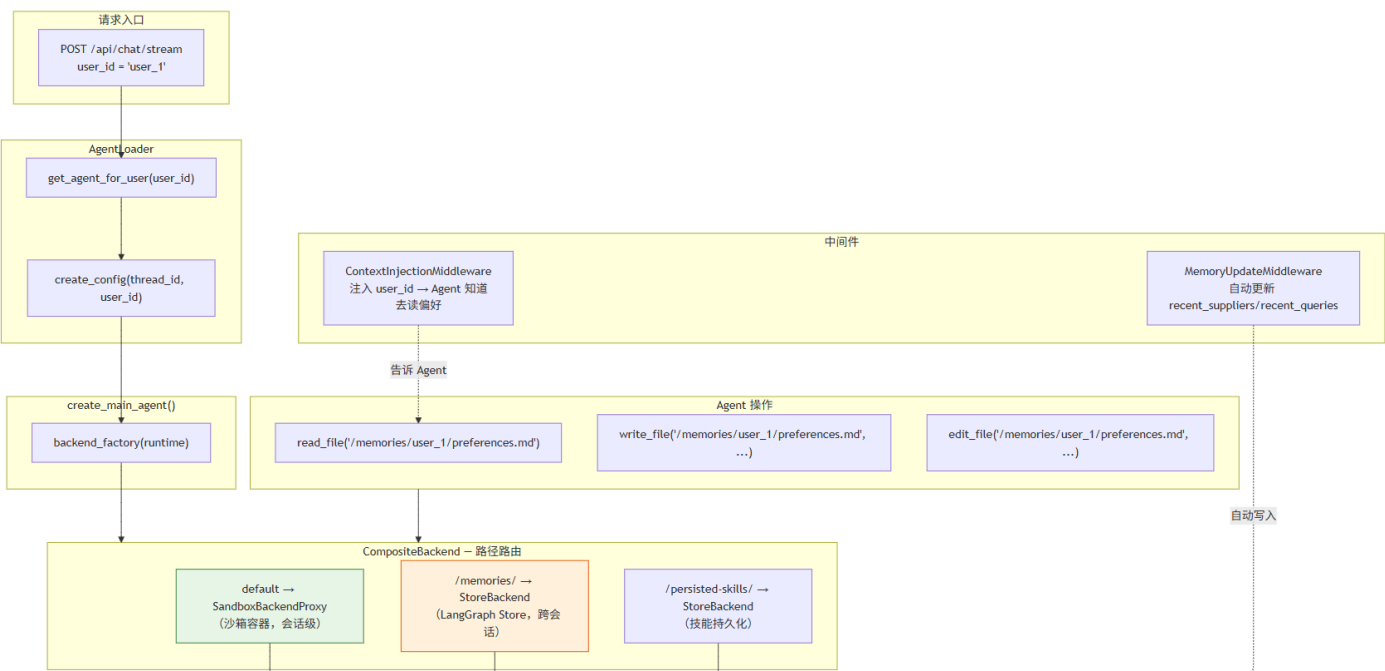
总结：9 个中间件形成 "健康守护 → 上下文注入 → 技能同步 → 摘要压缩 → 记忆更新 → 熔断保护 → 调用限制" 的完整生命周期管线。其中 #1 和 #7 协作形成沙箱两级防护，#3 和 #4 互补形成技能双通道，#5 和框架内置摘要形成被动/主动两层压缩。

3、用户偏好内容的长期记忆机制

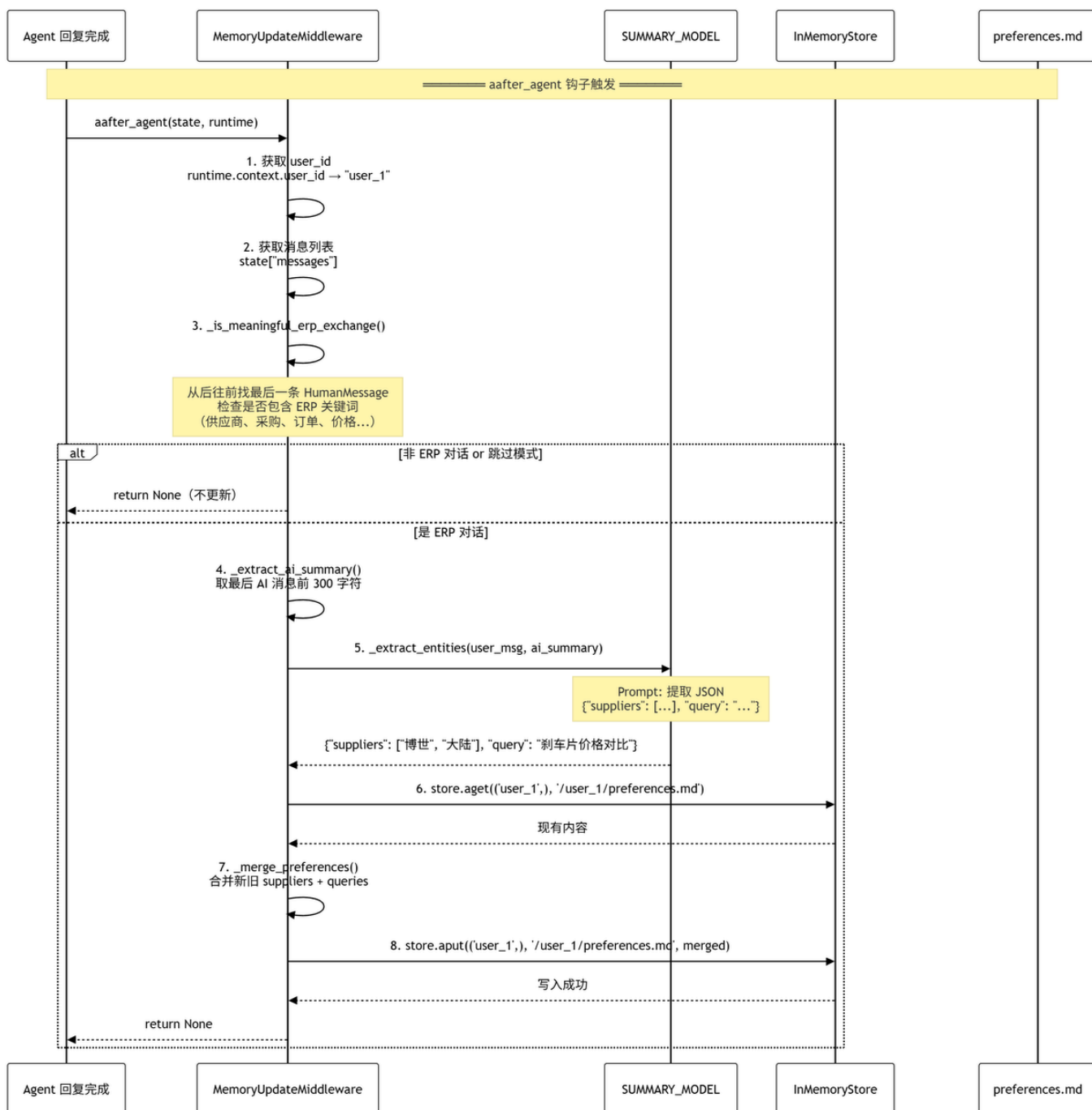
一、用户偏好记忆文件：preferences.md 的生命周期



二、用户偏好记忆文件的整体流程



三、用户偏好内容的更新机制

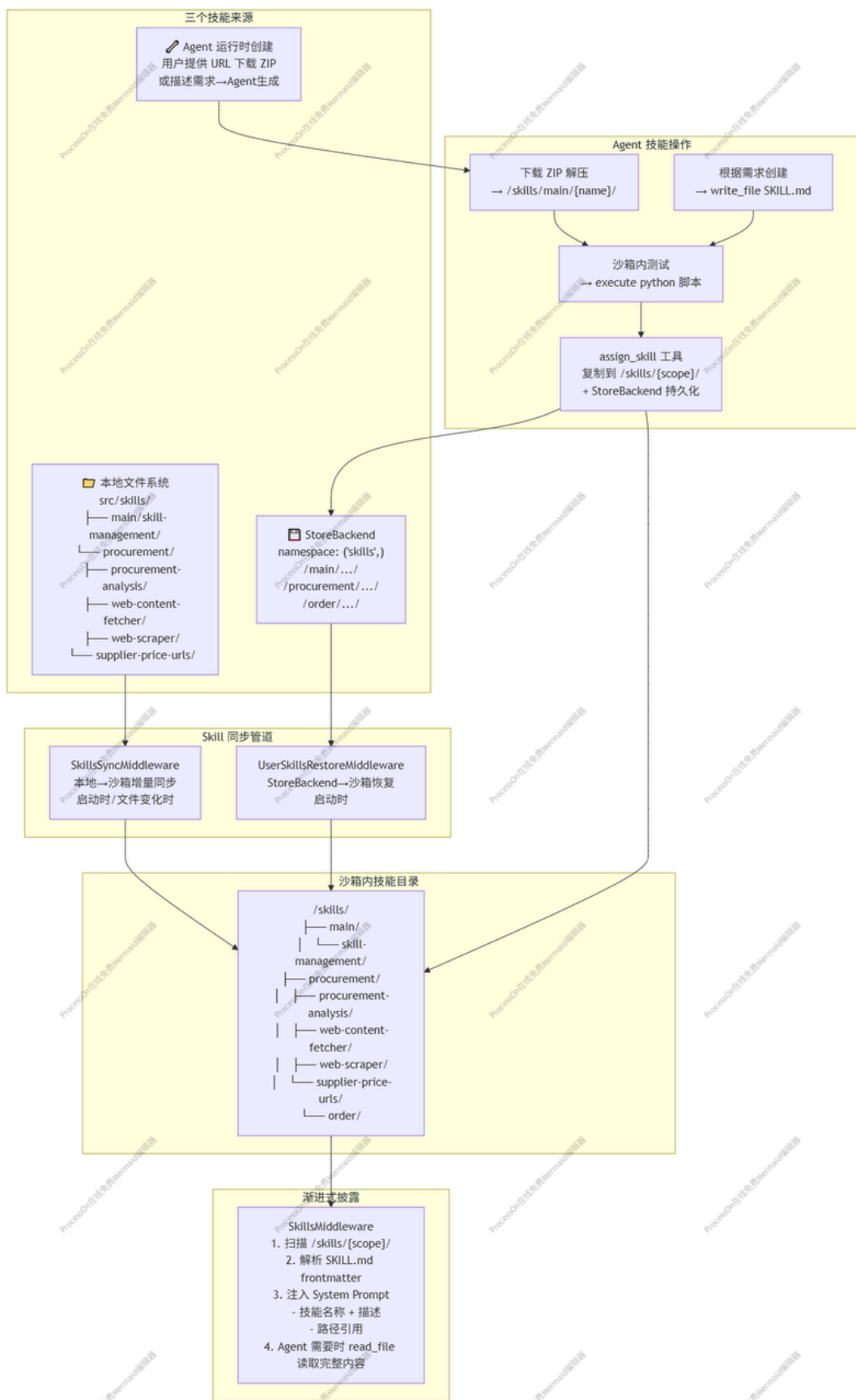


4、Skills的进化管理机制

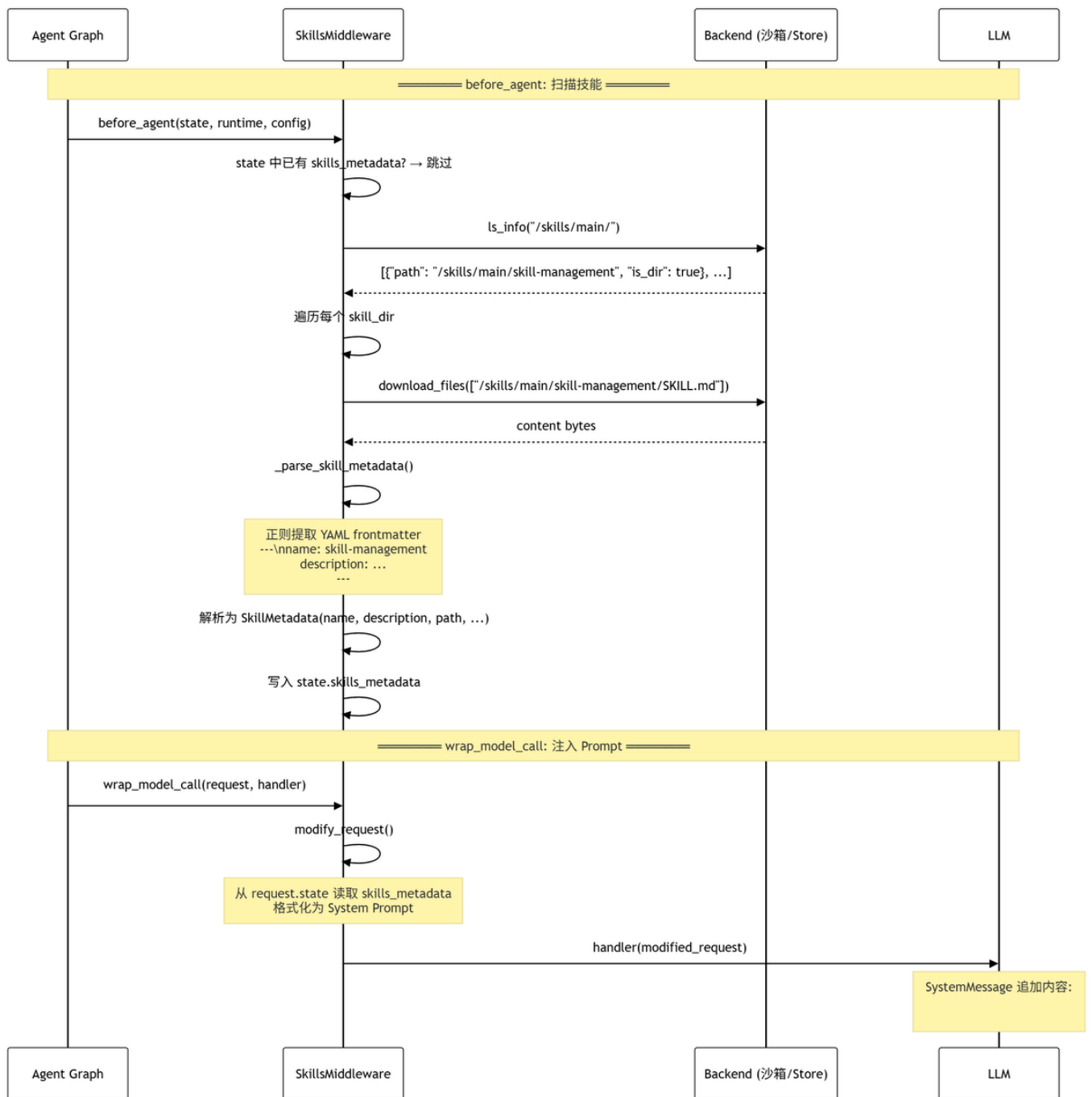
一、相关组件

组件	角色	输入	
SkillsMiddleware (中间件)	渐进式披露引擎	Backend 中的技能目录	System Pror
SkillsSyncMiddleware (自定义)	预置技能同步管道	src/skills/ 本地文件	沙箱 /skills/
UserSkillsRestoreMiddleware (自定义)	持久化技能恢复管道	StoreBackend ('skills',)	沙箱 /skills/
assign_skill 工具 (自定义)	技能分配 + 持久化	沙箱 /skills/main/{name}/	沙箱 /skills/{
download_skill.py 脚本	ZIP 下载 + 解压	URL	沙箱 /skills/r
skill-management SKILL.md	Agent 操作手册	—	5 阶段工作流
SCOPE_MAP	Agent→目录映射	agent_name	scope 目录名
CompositeBackend	存储路由器	路径	SandboxBac
InMemoryStore	持久化存储引擎	store API 调用	内存中的 key

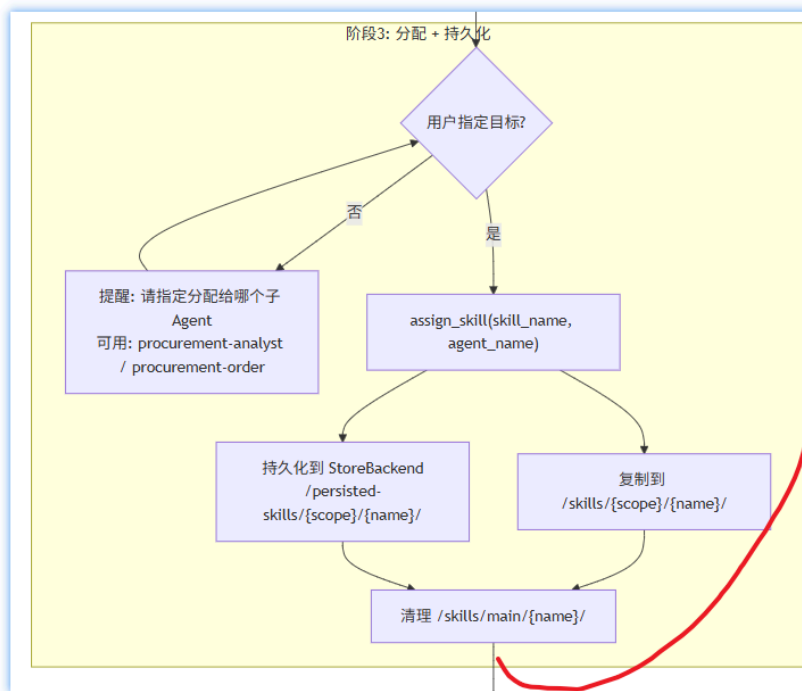
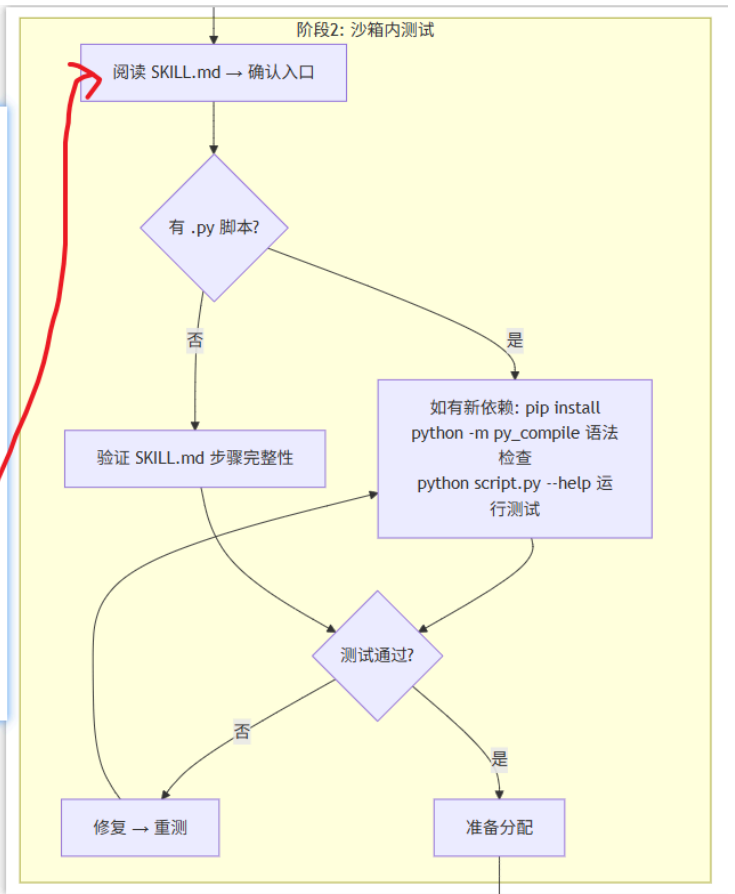
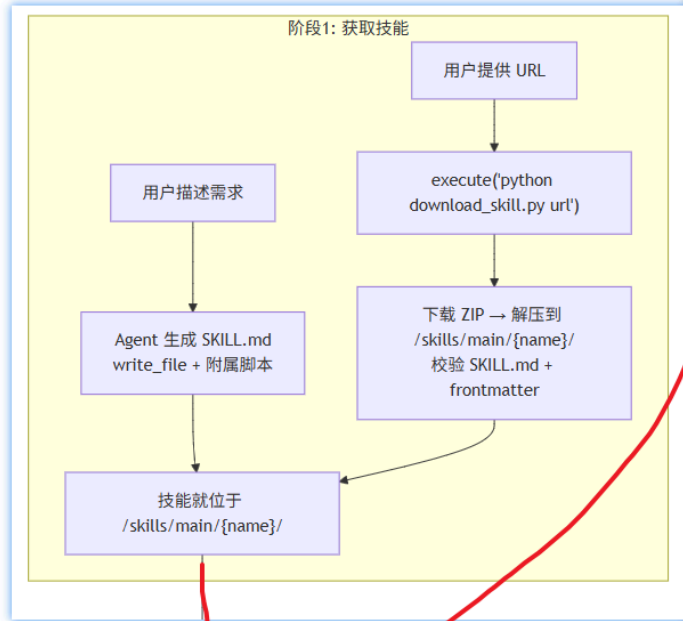
二、Skills的总体流程



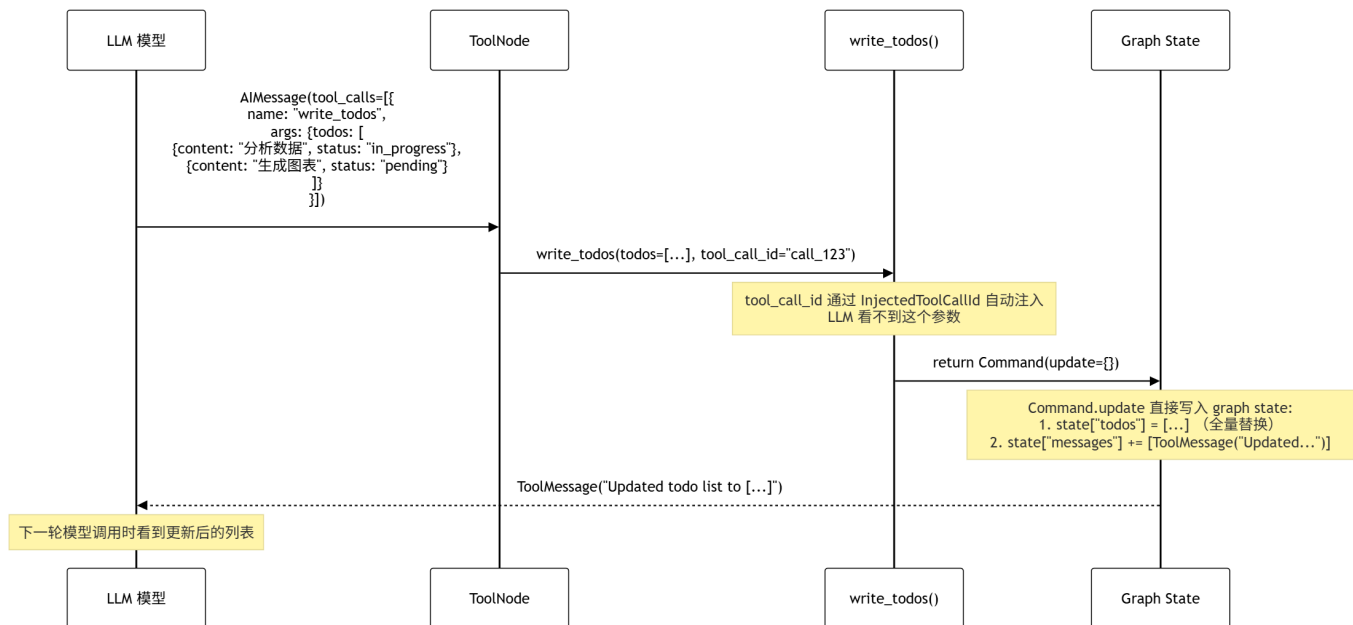
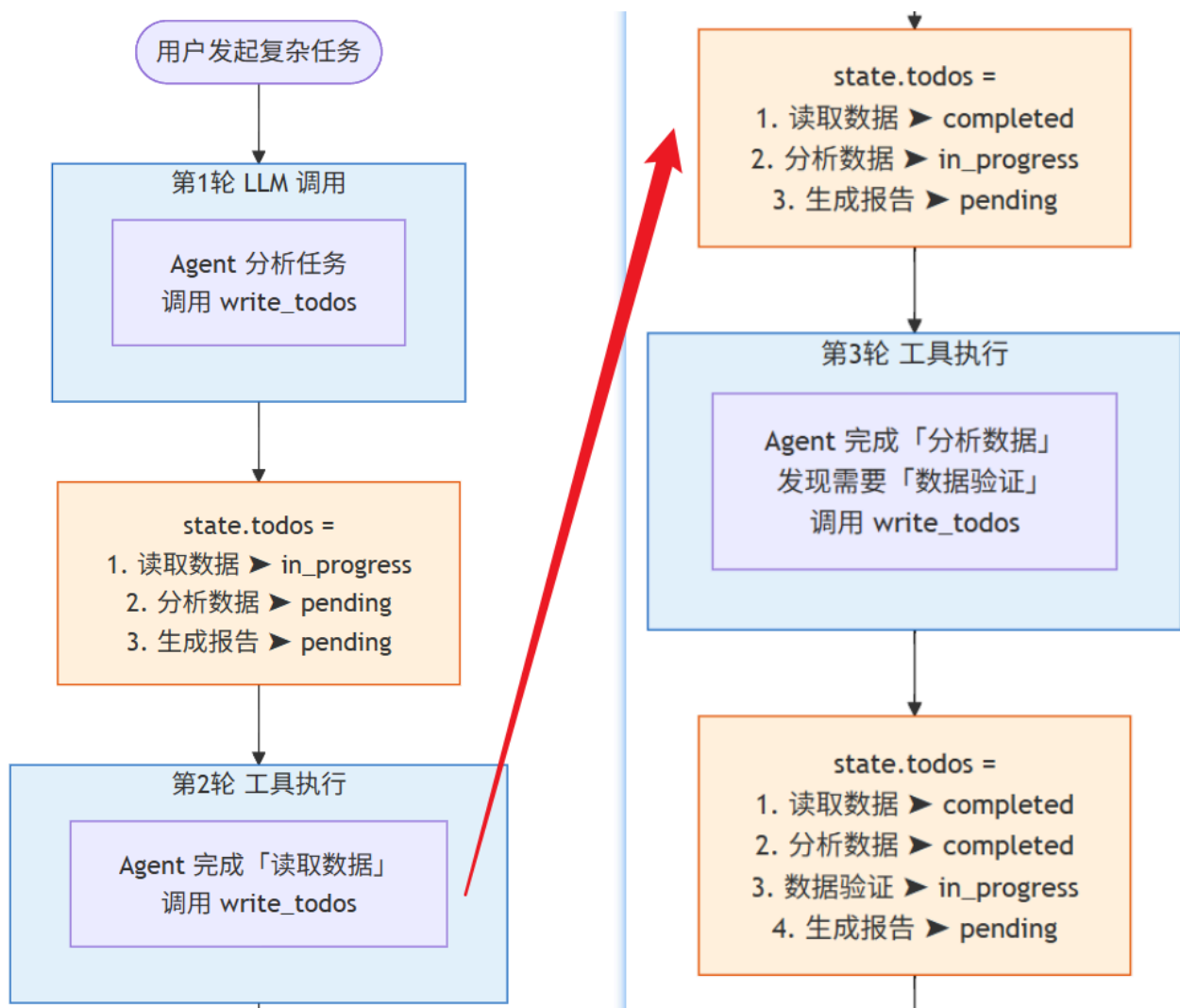
三、Skills的渐进式披露机制



四、Skills的自我进化机制（自动下载，自动创建，自动分配）



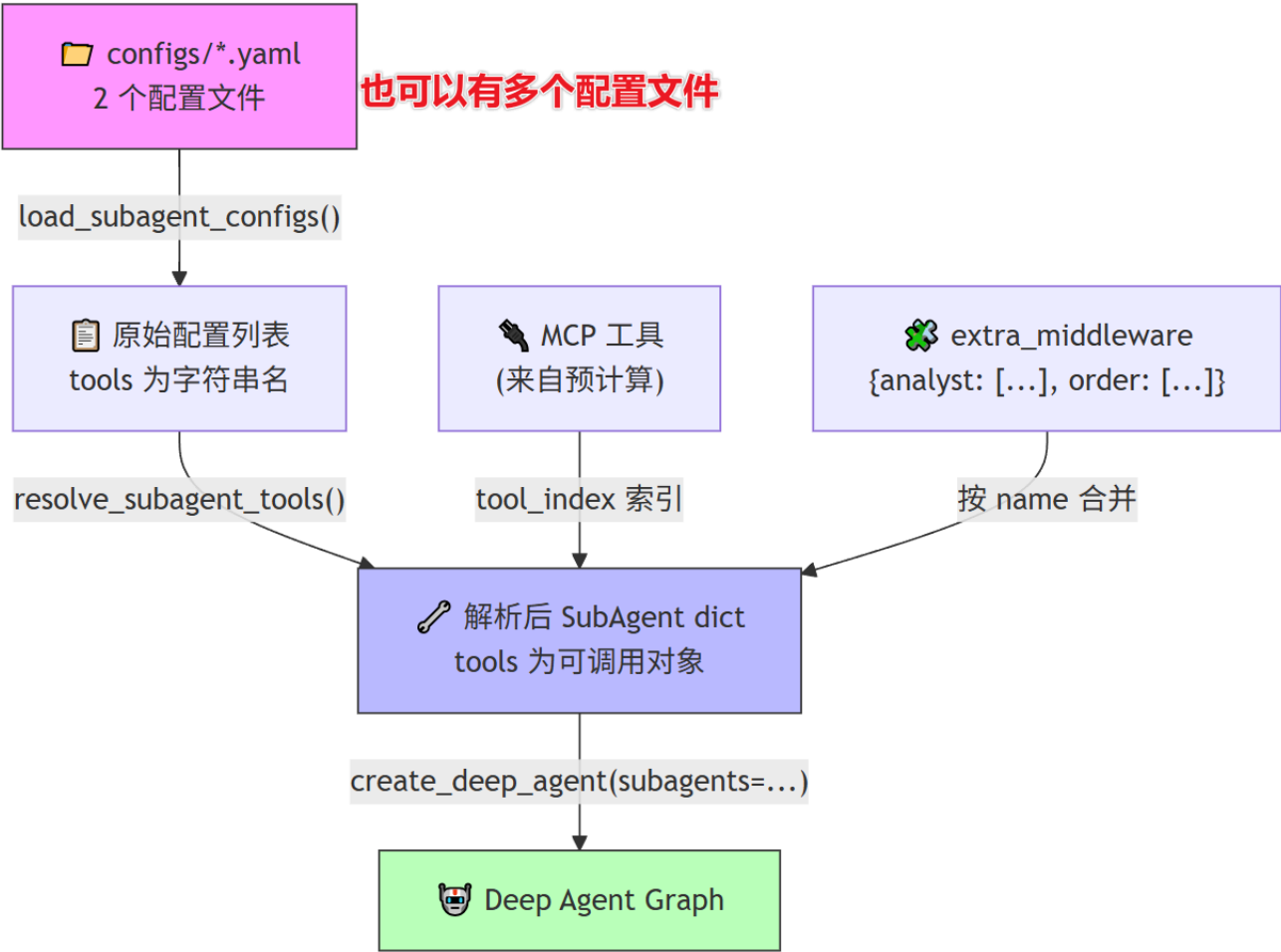
5、复制任务的Planning能力



6、SubAgent的弹性加载机制

项目中已有的，两个子 Agent 概览

维度	procurement-analyst	procur
职责	供应商比价、行情分析、策略建议、市场调研	创建/修改采购单、订单
触发关键词	分析、对比、报告、建议、评估、行情、比价、筛选	下单、创建订单、修改采
工具数	7 个（6 MCP + web_search + visualization）	5 个（3 MCP + web_sea
技能路径	/skills/procurement/	/skills/order/
中间件	SummarizationTool + 调用限制(50/200)	调用限制(20/50)，无摘要
特殊机制	26 种图表统一入口、沙箱内 web-scraper	HITL 人工审批（interru



- 第 1 步：`load_subagent_configs()` — 读取 `src/agent/subagents/configs/*.yaml`，校验 4 个必填字段（name、description、system_prompt、tools），返回原始 dict 列表，此时 tools 仍是字符串名称。
- 第 2 步：`resolve_subagent_tools()` — 将 tools 字符串通过子串匹配（`pattern in tool_name`）映射为实际工具对象，同时合并 `extra_middleware`。
- 第 3 步：传入 `create_deep_agent(subagents=...)` — DeepAgents 框架接管，每个 SubAgent 成为一个独立的 agent node。

扩展新子 Agent 的模式：

只需增加一个YAML文件，无需改任何 Python 代码：

在 `src/agent/subagents/configs/` 下新建 `new_agent.yaml`：

代码块

```
1  name: my-agent
2  description: >
3    我的子 Agent 描述，包含触发关键词。
4  tools:
5    - some_mcp_tool
6    - web_search
7  skills:
8    - /skills/my-domain/
9  system_prompt: |
10    你是 XXX 助手...
```

第四、基于SSE事件流的完整的前后端

1、架构概览：三文件分工

代码块

```
1  api_view/api/
2  |—— chat.py      ★ 核心：SSE 流式对话 + 中断检测 + 中断恢复 + 展示消息持久化
3  |—— history.py   ← 历史会话列表 / 消息查询 / 会话删除
4  |—— agent_loader.py (上层) ← Agent 单例 / MongoDB 连接 / 展示消息存取
```

2、双流模式 — 一条连接同时走两路

chat.py 从头到尾只做一件事：把 Agent 流式输出转换成前端能消费的 SSE 事件序列。但它的实现方式体现了项目特色。

代码块

```
1  # chat.py:281-287
2  async for chunk in agent_loader.agent.astream(
3      input=current_input,
4      config=config,
5      stream_mode=["messages", "values"], # ← 双模式
6      subgraphs=True,
```

```
7     version="v2",
8 ):
```

这是整个中断检测机制的基础。常规 SSE 只用 "messages" 模式拿 token/tool 事件，但本项目需要检测中断，所以同时开了 "values"：

代码块

```
1 stream_mode=["messages", "values"]
2     |
3     |    └→ values 流: 检测 interrupts() 是否触发
4     |    chunk_type == "values" && chunk.get("interrupts")
5     |    先于 messages 处理 (代码中 if 在 messages 前面)
6     |
7     └→ messages 流: token / tool_call / tool_result
8         chunk_type == "messages"
9         正常 SSE 事件 (token / tool_start / tool_args / tool_result /
            tool_end)
```

流 A: `messages` — 内容输出 (AIMessage, ToolMessage)

代码块

```
1 messages 流产出的 token 类型:  
2  
3 [-----]  
4 | token.tool_call_chunks 有值                                | - AIMessage (调用工  
   | 具的指令，一般没有content)                               |  
5 | → token.tool_call_chunks[i]["name"] = "web_search"        | → tool_start 事件  
6 | → token.tool_call_chunks[i]["args"] = '{"query":...}'      | → tool_args 事件  
7 |                                                            |  
8 | token.type == "tool"                                       | # ToolMessage (工  
   | 具调用后返回的内容)                                       |  
9 | → token.name = "web_search"                                 | → tool_result 事件  
10| → token.content = "搜索结果是..."                         | → tool_end 事件  
11|                                                            |  
12| token.content 是纯文本 (不是 tool, 不是 tool_call_chunks)|  
13| → "根据搜索结果，博世刹车片..."                          | → token 事件  
14[-----]
```

流 B: values — 中断检测

代码块

```
1  # 中断检测必须在 messages 处理之前执行 (line 296-366)
```

```

2  if chunk_type == "values" and chunk.get("interrupts"):
3      for interrupt in chunk["interrupts"]:
4          if "action_requests" in interrupt.value:
5              → interrupt_type = "hit_approval" # 第2层: 审批中断
6          elif interrupt.value.get("type") == "order_info_request":
7              → interrupt_type = "order_info_supplement" # 第1层: 数据补充
8      → 保存 display_messages 到 MongoDB
9      → 发送 done(interrupted=True)
10     → return # 停止本次流

```

为什么必须先检测中断？ 因为中断时最后一条 messages 事件可能还来不及到达，先处理中断才能确保两端状态一致。

3、完整请求链路（从用户输入到 SSE 返回）

代码块

```

1  用户发送 "帮我新增采购订单"
2  |
3  ▼
4  POST /api/chat/stream {message: "帮我新增采购订单"}
5  |
6  ▼
7  chat_stream() → StreamingResponse
8  |
9  ▼
10 stream_chat_response(message="帮我新增采购订单")
11 |   config = agent_loader.create_config(thread_id)
12 |   current_input = {"messages": [{"role": "user", "content": "..."}]}
13 |   display_messages = [user消息]
14 |
15 ▼
16 agent.astream(input, config, stream_mode=["messages", "values"],
17 subgraphs=True, version="v2")
18 |
18 |→ 主 Agent 收到消息 → 判断: 订单操作 → task(procurement-order)
19 |
20 |→ [SSE] token: "好的, 我来帮您..." (source: main)
21 |→ [SSE] tool_start: task (source: main)
22 |→ [SSE] tool_args: {...} (source: main)
23 |
24 |→ order 子 Agent 启动 (独立上下文)
25 |   |→ 第1步: 提取数据
26 |   |→ 第2步: Schema 校验 → partId 缺失!

```

```

27 |     |→ 第3步: request_order_info(...)
28 |     |    |→ interrupt({"type": "order_info_request", ...})
29 |     |
30 |→ [stream_mode="values"] 检测到 interrupts
31 |     |
32 |     |→ [SSE] interrupt {interrupt_type: "order_info_supplement", ...}
33 |     |→ save_display_messages(thread_id, cleaned) ← 保存现场
34 |     |    |→ [SSE] done {interrupted: true}
35 |         return ← 流结束
36 |
37 ▼
38 前端 InterruptBanner 弹出: "请输入补充信息"
39 用户输入: "物料ID=100, 数量=50, 单价=25.5"
40 |
41 ▼
42 POST /api/chat/{thread_id}/resume {resume: {supplement: "物料ID=100..."}}
43 |
44 ▼
45 stream_chat_response(thread_id, resume_data={supplement: "..."})
46 | existing = get_display_messages() ← 加载中断前的消息
47 | current_input = Command(resume={supplement: "..."}) ← 恢复
48 |
49 ▼
50 agent.astream(Command(resume=...), config, ...)
51 |
52 |→ order 子 Agent 恢复 → 解析补充数据 → 合并 → 校验通过
53 |→ 构造 order_create 参数 → interrupt_on 触发
54 |
55 |→ [stream_mode="values"] 再次检测到 interrupts
56 |     |→ [SSE] interrupt {interrupt_type: "hitl_approval",
action_requests: [...]}
57 |     |    |→ [SSE] done {interrupted: true}
58 |
59 ▼
60 前端 InterruptBanner 切换: 审批卡片 (approve/reject)
61 用户点击 "approve"
62 |
63 ▼
64 POST /api/chat/{thread_id}/resume {resume: {decisions: [{type: "approve"}]}}
65 |
66 ▼
67 agent.astream(Command(resume={decisions:...}), config, ...)
68 |
69 |→ order_create 执行 → 成功
70 |→ [SSE] tool_result {text: "订单创建成功..."}
71 |→ [SSE] token "订单已创建, 编号 P020260513..." (source: order)
72 |

```

```
73    └→ 主 Agent 收到子Agent 完成 → compact_conversation
74    └→ [SSE] token "已为您完成订单创建..."          (source: main)
75    |
76    └→ save_display_messages(thread_id, all_messages)
77    [SSE] done {thread_id: "...", content: "..."}
```